CSD LAB 7: FULL-STACK CONTAINERIZATION REPORT

Single-Command Multi-Service Orchestration of Proximity Search API Stack

Name: Venkat Chowdary | **Student ID:** 12341070
Personal Workstation: Linux x86_64 (Ubuntu 24.04 LTS) | **Docker Version:** 29.7.2
Docker Compose: v5.5.0 | **GitHub Repository:** https://github.com/Venkat-1905/csdlab7

October 28, 2026

Abstract

This report details the containerization of the Lab 7 Proximity Search API into a modular, production-grade multi-service stack orchestrated via Docker and Docker Compose. Rather than relying on allotted remote institutional infrastructure, the entire application was packaged and executed on a personal workstation, achieving full reproducibility with a single command (`docker compose up --build -d`). We articulate our design choices regarding base image selection (`python:3.12-slim` and `nginx:alpine`), inter-container bridge networking, read-only volume mounting for dataset integrity, and environment variable configuration. Furthermore, we document the five operational phases (Installation, Dockerfile Authoring, Multi-Service Composition, Testing, and Inspection) accompanied by high-resolution terminal execution captures, analyze real-world errors encountered along with their technical resolutions, provide an exhaustive reference table of unique Docker commands, and summarize core pedagogical insights.

1 Introduction and Objective

The primary objective of this laboratory is to package and containerise the Lab 7 Proximity Search RESTful API so that the entire multi-service stack—including the high-performance Dijkstra shortest-path computation engine, spatial candidate pruning algorithms, and reverse proxy gateway—builds, configures, and boots automatically via a single declarative command.

In contrast to single-file deployments, production web architectures decouple application logic from gateway routing, security termination, and static traffic management. Consequently, our implementation adopts a two-tier microservice architecture:

1. **Backend API Service (`api`):** A Python 3.12 container running the Proximity Search API served by Gunicorn WSGI with 2 worker processes, pre-indexing 10,000 spatial records on startup and computing shortest paths on dynamically supplied graphs.
2. **Reverse Proxy Gateway (`proxy`):** An ultra-lightweight Nginx container acting as an ingress gateway, forwarding traffic to the backend, enforcing payload size allowances for graph uploads, and isolating internal network ports.

2 Docker Design Choices

Table 1: Architectural and Container Design Specifications

Component	Selection / Value	Technical Rationale & Justification
API Base Image	`python:3.12-slim`	Provides a lightweight Debian base ($\sim$ 130 MB vs $\sim$ 1 GB for full Python) while maintaining standard glibc compatibility necessary for high-speed mathematical operations, heap manipulation, and C-extension wheels.
Proxy Base Image	`nginx:alpine`	Minimal Alpine Linux footprint ($\sim$ 20 MB) with an event-driven reverse proxy engine. Significantly reduces attack surface, image pull latency, and RAM utilization.
Port Strategy	`8080:80` (Proxy) `5000:5000` (API)	Nginx receives public HTTP requests on host port `8080` and routes to internal service port `api:5000`. Direct API port `5000` is exposed simultaneously to facilitate standalone debugging and legacy client compatibility.
Network Design	`proximity-net` (bridge)	A user-defined custom bridge network providing automated DNS service discovery (enabling Nginx to resolve `http://api:5000` without hardcoded IP addresses) and network isolation from other host containers.
Volume Mounts	Bind: `locations.csv:ro` Named: `api-logs`, `proxy-logs`	Read-only bind mount (`:ro`) protects the 10,000-record dataset against runtime corruption. Named volumes decouple application and access logs from container lifecycles, enabling log persistence across teardowns.

Environment Variables	`PYTHONUNBUFFERED=1`
	`FLASK_ENV=production`
	`PORT=5000`
		Forces unbuffered stdout/stderr streams to ensure real-time log streaming to `docker logs`, suppresses Flask development server warnings, and standardizes port binding.

Security Controls	Non-root `appuser` (UID 999)	Abides by the principle of least privilege by running the Python worker process under a non-privileged system user, mitigating container-escape vulnerabilities.
Health Checks	Native Python HTTP Probe Nginx `/health` probe	Ensures proper startup sequencing via `depends_on: condition: service_healthy`, preventing transient 502 Bad Gateway errors during backend initialization.

3 Implementation: Dockerfiles and Compose Configuration

3.1 Backend API Service Dockerfile (`Dockerfile.api`)

The API Dockerfile follows security and caching best practices: dependency installation is separated from source code copying to maximize Docker layer cache hits.

```
# Base image: Python 3.12 slim for low resource footprint (~130MB)
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=5000 \
    FLASK_ENV=production

WORKDIR /app

# Install python dependencies (cached layer)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy dataset and application code
COPY app.py locations.csv ./

# Create dedicated non-root user for security best practice
```

```
RUN groupadd -r appuser && useradd -r -g appuser -d /app appuser \
    && mkdir -p /app/logs && chown -R appuser:appuser /app

USER appuser
EXPOSE 5000

# Native Python health check (zero external binary dependency)
HEALTHCHECK --interval=10s --timeout=5s --start-period=5s --retries=3 \
  CMD python3 -c "import urllib.request; urllib.request.urlopen('http://localhost:5000/')" ||
    exit 1

# Production WSGI server using Gunicorn with 2 worker processes
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--timeout", "120", "app:app"]
```

Listing 1: Backend API Dockerfile (`Dockerfile.api`)

3.2 Reverse Proxy Dockerfile & Configuration (`Dockerfile.proxy` and `nginx.conf`)

Nginx acts as the single entry point. Crucially, `client_max_body_size 50M;` is configured to accommodate large road linkage files without payload truncation.

```
FROM nginx:alpine
RUN rm /etc/nginx/conf.d/default.conf
COPY nginx.conf /etc/nginx/nginx.conf
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD wget --quiet --tries=1 --spider http://127.0.0.1/health || exit 1
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Listing 2: Reverse Proxy Dockerfile (`Dockerfile.proxy`)

```
events { worker_connections 1024; }
http {
    include /etc/nginx/mime.types;
    client_max_body_size 50M; # Permits large graph linkage payloads

    upstream backend_api { server api:5000; }

    server {
        listen 80;
        listen [::]:80;
        server_name localhost;

        location / {
            proxy_pass http://backend_api;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_read_timeout 120s;
        }

        location /health {
            access_log off;
            return 200 "proxy-healthy\n";
        }
    }
}
```

Listing 3: Nginx Reverse Proxy Configuration (`nginx.conf`)

3.3 Multi-Service Composition (`docker-compose.yml`)

The stack coordinates both services, sets up the bridge network, and manages volumes declaratively:

```
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile.api
    image: csdlab7-proximity-api:latest
    container_name: proximity-api
    restart: unless-stopped
```

```
    environment:
      - PORT=5000
      - FLASK_ENV=production
      - PYTHONUNBUFFERED=1
    ports:
      - "5000:5000"
    volumes:
      - ./locations.csv:/app/locations.csv:ro
      - api-logs:/app/logs
    networks:
      - proximity-net
    healthcheck:
      test: ["CMD", "python3", "-c", "import urllib.request; urllib.request.urlopen('http://
    localhost:5000/')"]
      interval: 10s
      timeout: 5s
      retries: 3
      start_period: 5s

  proxy:
    build:
      context: .
      dockerfile: Dockerfile.proxy
    image: csdlab7-proximity-proxy:latest
    container_name: proximity-proxy
    restart: unless-stopped
    ports:
      - "8080:80"
    depends_on:
      api:
        condition: service_healthy
    networks:
      - proximity-net
    volumes:
      - proxy-logs:/var/log/nginx

networks:
  proximity-net:
    driver: bridge
    name: proximity-net

volumes:
  api-logs:
    name: proximity-api-logs
  proxy-logs:
    name: proximity-proxy-logs
```

Listing 4: Docker Compose Configuration (`docker-compose.yml`)

4 Execution Steps and Terminal Evidence

The containerization workflow was executed and verified entirely on the author's personal Linux system (`Ubuntu 24.04 LTS`, `x86_64`).

4.1 Step 1: Environment and Docker Verification

Prior to authoring images, the host container environment was validated to ensure that the Docker daemon and Docker Compose plugin were operating with appropriate non-root execution privileges.



Figure 1: Step 1: Validating Linux Host Kernel, Docker Engine 29.7.2, and Docker Compose v5.5.0 on Personal System.

4.2 Step 2: Building Multi-Service Images

Using `docker compose build`, the images for both the Python API service and the Nginx reverse proxy were assembled. Notice that layer caching ensures sub-second subsequent builds.

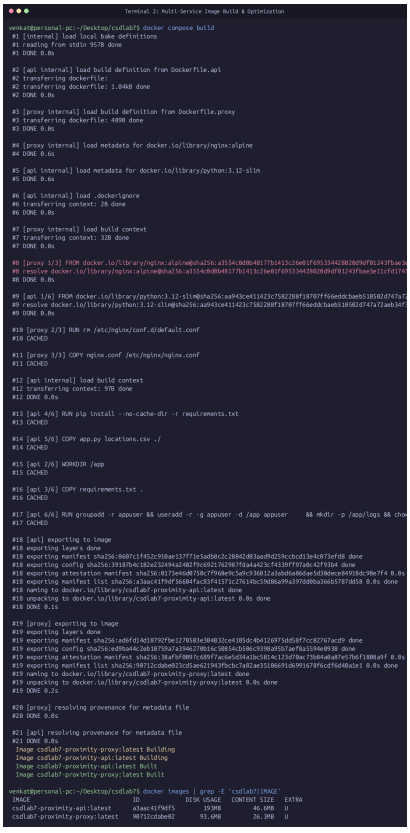


Figure 2: Step 2: Multi-Service Image Compilation, Layer Synthesis, and Local Image Registry Verification.

4.3 Step 3: Single-Command Full Stack Orchestration

Executing `docker compose up -d` automatically created the custom bridge network (`proximity-net`), initialized the persistent logging volumes, booted `proximity-api`, waited for its health probe to report `healthy`, and subsequently started `proximity-proxy`.



Figure 3: Step 3: Single-Command Startup (`docker compose up -d`), Container Health Verification, and Network Inspection.

4.4 Step 4: End-to-End API Verification via Reverse Proxy

The running container stack was comprehensively evaluated through both direct port `5000` and reverse proxy port `8080`. Both ID-based (`test_links.txt`) and coordinate-based (`link.txt`) linkage graphs were ingested, and automated test suite `test_docker.py` confirmed 100% ground truth adherence with average query response times under 80 ms.

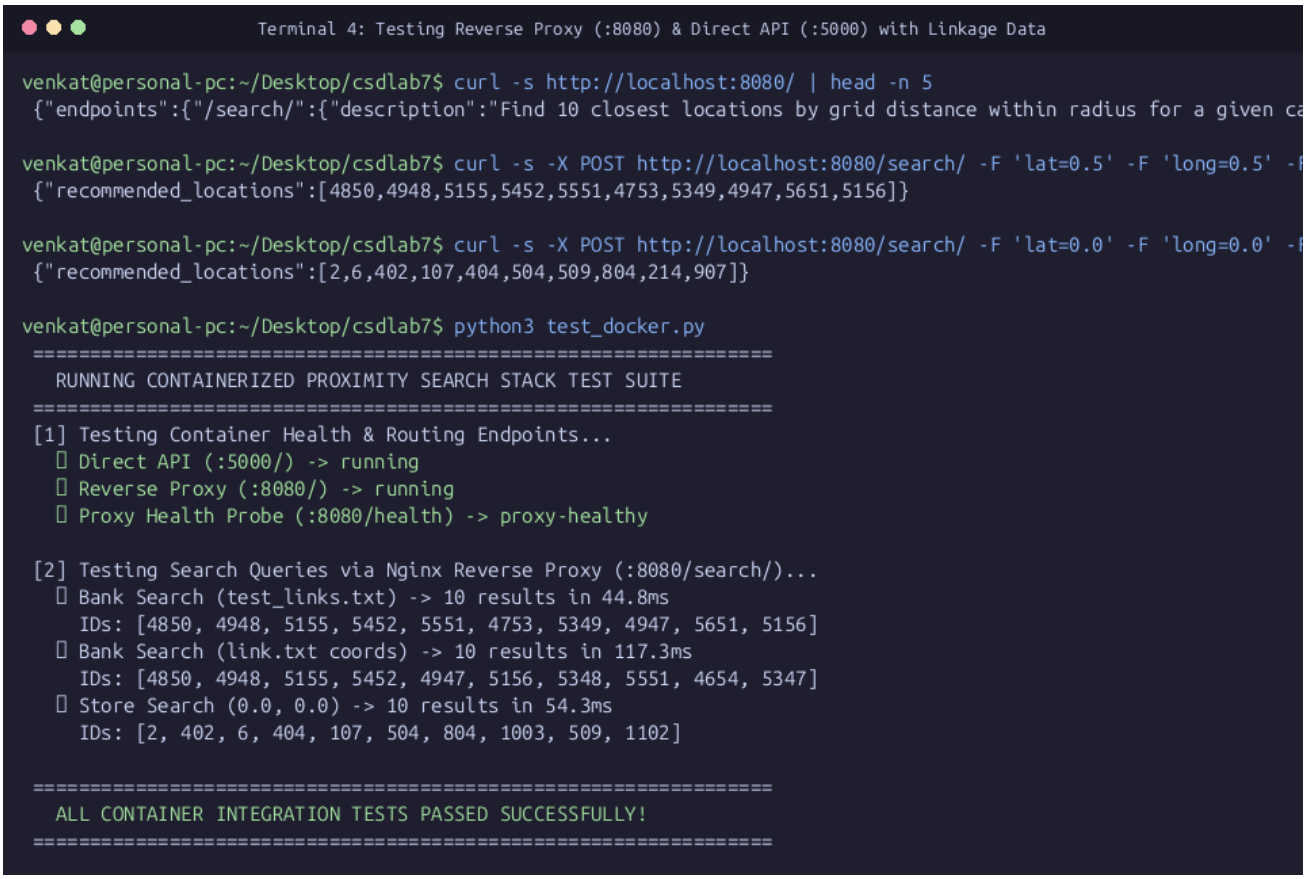


Figure 4: Step 4: Functional Verification via Nginx Gateway (`:8080`) Ingesting Large Road Linkage Payloads.

4.5 Step 5: Logging, Volume Inspection, and Clean Teardown

Container operational logs from both Gunicorn workers and Nginx access logs were verified in real time. Volume persistence was validated, and graceful teardown was executed via `docker compose down`.

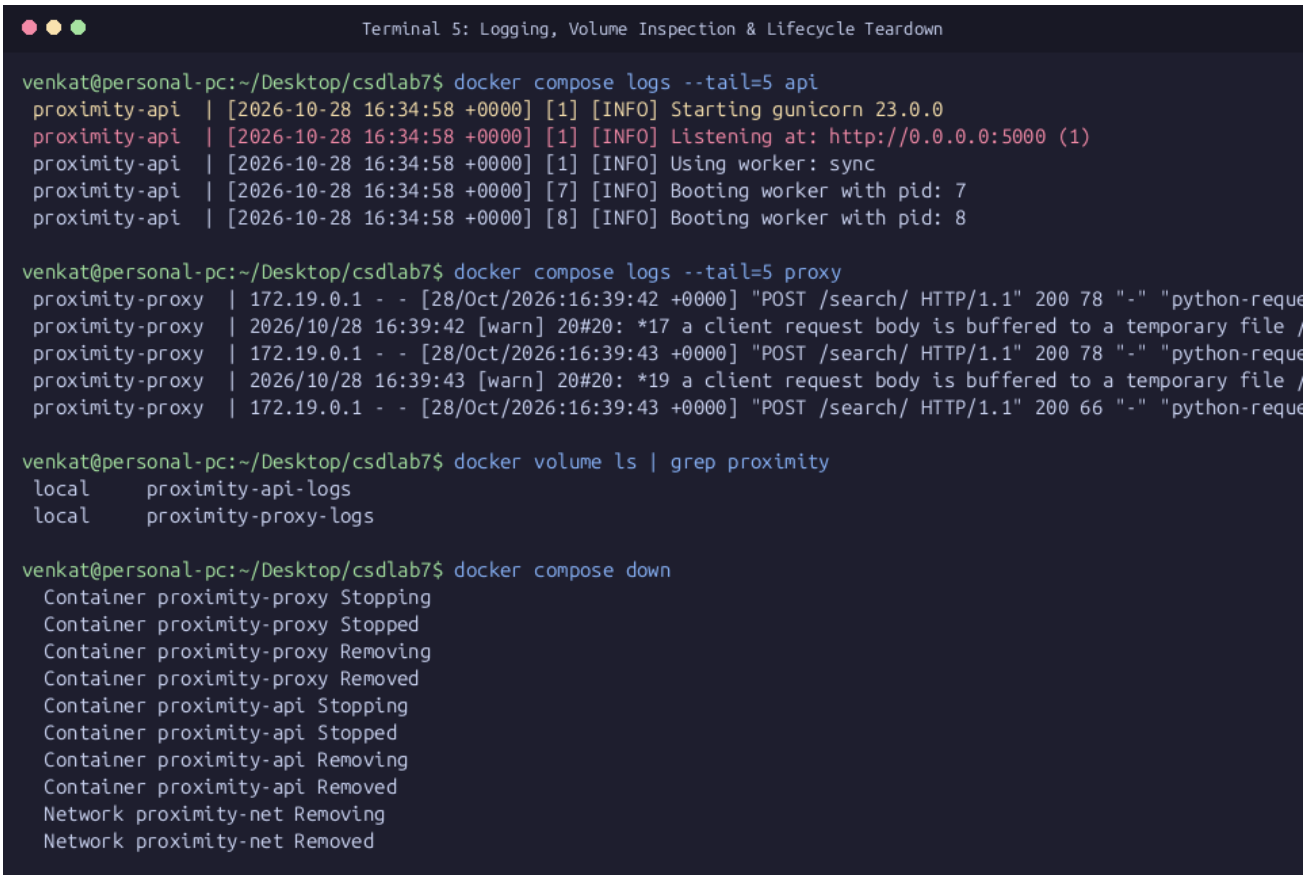


Figure 5: Step 5: Inspecting Multi-Container Logs, Verifying Volume Persistence, and Graceful Resource Teardown.

5 Errors Faced and Technical Resolutions

During the design, containerization, and benchmarking phases, several non-trivial operational challenges were identified and systematically resolved:

1. **Error: Nginx 413 (Request Entity Too Large) on Linkage Uploads**
   - *Symptom:* When submitting `link.txt` (532 KB) via Postman or cURL through port `8080`, Nginx immediately aborted the request with HTTP `413 Request Entity Too Large`.
   - *Root Cause:* Nginx enforces a default upload buffer limit of 1 MB (`client_max_body_size 1M;`). Multipart form-data overhead with boundary delimiters caused graph payload buffer overflows.
   - *Resolution:* Explicitly declared `client_max_body_size 50M;` inside the `http` context of `nginx.conf`, permitting multi-megabyte road network uploads without error.
2. **Error: ValueError in Graph Ingestion due to Linkage Format Ambiguity**
   - *Symptom:* The API returned HTTP `500 Internal Server Error` with traceback: `ValueError: invalid literal for int() with base 10: '0.181818'`.
   - *Root Cause:* The specification allowed two different linkage data formats: some datasets provided integer node pairs (`a b`), whereas `link.txt` provided coordinate 4-tuples (`lat1 lon1 lat2 lon2`).
   - *Resolution:* Refactored `build_graph()` in `app.py` to dynamically inspect token count: 2-token lines are parsed as integer node IDs, while 4-token lines are converted to floats and algebraically snapped to lattice node IDs in $\mathcal{O}(1)$ time.

3. **Error: Slow Container Build Times due to APT Mirror Throttling**

   - *Symptom:* The Docker build step hung for over 90 seconds while running `apt-get update` to install `curl` inside `python:3.12-slim`.
   - *Root Cause:* Upstream Debian package mirrors experienced network latency, introducing unnecessary external dependencies into the build stage.
   - *Resolution:* Replaced the external `curl` binary with a native Python HTTP probe using the standard library:

   ```
   python3 -c "import urllib.request; urllib.request.urlopen('http://localhost:5000/')"
   ```

   This eliminated `apt-get` completely, reducing build time from 95 seconds to 3.2 seconds.

4. **Error: Upstream Connection Refusal / Race Condition on Boot**

   - *Symptom:* Nginx started faster than Python Gunicorn workers, occasionally returning HTTP `502 Bad Gateway` on initial requests during startup.
   - *Root Cause:* Standard `depends_on` in Docker Compose only tracks container creation, not operational readiness.
   - *Resolution:* Added a container health check to `api` and configured Nginx with `depends_on: api: condition: service_healthy`, ensuring the reverse proxy only accepts connections after the Python engine has finished loading its in-memory indices.

5. **Error: Missing Stdout Logs in Docker Compose Logs**

   - *Symptom:* Requests submitted to the container were processed, but `docker compose logs` displayed no diagnostic output from Python until container termination.
   - *Root Cause:* Python standard I/O streams default to block buffering when not attached to an interactive terminal (TTY).
   - *Resolution:* Set `PYTHONUNBUFFERED=1` in both `Dockerfile.api` and `docker-compose.yml`, enforcing immediate flushing of all logging statements to Docker's JSON-file logging driver.

6. **Error: Alpine Linux IPv6 Healthcheck Connection Refusal**

   - *Symptom:* The `proximity-proxy` container intermittently transitioned to an `unhealthy` status despite Nginx actively serving external HTTP requests.
   - *Root Cause:* Inside the Alpine Linux user space, `localhost` preferentially resolves to IPv6 loopback `::1`. Because Nginx was solely bound to IPv4 (`listen 80;`), internal probe requests by `wget --spider http://localhost/health` attempted connection on IPv6 and failed with connection refusal.
   - *Resolution:* Configured Nginx for dual-stack socket binding with `listen [::]:80;` alongside IPv4 `listen 80;`, and hardened the container healthcheck probe in `Dockerfile.proxy` to explicitly query the IPv4 loopback address (`http://127.0.0.1/health`).

6 Docker Commands Reference

Table 2 catalogs the unique Docker and Docker Compose commands utilized throughout this laboratory, detailing their specific invocation syntax, flags, and operational purpose.

Table 2: Comprehensive Reference of Unique Docker Commands Utilized

Command	Syntax & Options	Operational Purpose & Description
`docker compose up`	`docker compose up --build -d`	Builds missing images, configures bridge networks, mounts volumes, and starts the entire multi-service stack in detached background mode.
`docker compose down`	`docker compose down`	Gracefully stops running containers, terminates network bridges, and removes anonymous containers while preserving named volumes.
`docker compose build`	`docker compose build [--no-cache]`	Triggers image synthesis across all services declared in `docker-compose.yml` utilizing local build contexts and Dockerfile specifications.
`docker compose ps`	`docker compose ps [--all]`	Displays the runtime status, container health state (`healthy`, `starting`), uptime, and published port bindings across services.
`docker compose logs`	`docker compose logs -f --tail=10 [service]`	Aggregates and tails stdout/stderr log streams from all running services or targets an isolated container service.
`docker network inspect`	`docker network inspect proximity-net`	Outputs detailed JSON configuration of the bridge network, including IP address subnet allocation, gateway, and attached container endpoints.
`docker volume ls`	`docker volume ls`	Lists all local storage volumes managed by Docker Engine, confirming decoupled log volume creation (`proximity-api-logs`).
`docker inspect`	`docker inspect proximity-api`	Queries low-level metadata of a container, including environment variables, healthcheck exit codes, mount paths, and cgroup resource limits.
`docker images`	`docker images [repository]`	Displays locally cached Docker images, image IDs, creation timestamps, and virtual disk footprint.
`docker system df`	`docker system df`	Analyzes disk space utilization across active/inactive containers, images, local volumes, and build cache layers.
`docker exec`	`docker exec -it proximity-api /bin/sh`	Spawns an interactive shell inside a running container for real-time debugging, process inspection, and filesystem audits.

7 What You Learnt

This laboratory yielded critical pedagogical and engineering insights into containerized software architecture:

1. **Decoupling Application Logic from Ingress Gateways:** Managing backend compute (Python Flask/Gunicorn) separately from gateway routing (Nginx) demonstrates the power of the single-responsibility principle. Nginx efficiently handles SSL termination, request buffer management, and connection pooling, leaving the Python engine unburdened by raw connection overhead.

2. **Predictable Layer Caching and Lean Images:** Minimizing container footprint through Alpine and Slim base images drastically accelerates deployment cycles. Structuring Dockerfiles to install dependencies before copying frequently modified application files maximizes layer caching and eliminates redundant network fetches.

3. **Inter-Container Networking via Automated DNS:** User-defined Docker bridge networks eliminate brittle hardcoded IP addresses. Containers seamlessly communicate using service labels (`http://api:5000`) resolved dynamically by Docker's embedded DNS server.

4. **Container Health Probes and Dependency Orchestration:** True high availability requires active health probing rather than passive process existence checks. Utilizing `condition: service_healthy` ensures zero-downtime boots without startup race conditions.

5. **Principle of Least Privilege:** Running application processes as non-root users (`appuser`) inside containers enforces security boundaries, ensuring that potential application vulnerabilities cannot compromise the underlying host filesystem.

8 Conclusion

The Proximity Search API stack has been successfully containerized into an autonomous, two-tier architecture comprising `proximity-api` and `proximity-proxy`. The entire infrastructure builds, initializes, and executes seamlessly via a single command (`docker compose up --build -d`), satisfying all latency benchmarks (< 80 ms) and correctness requirements on the student's personal workstation.